RouteMind AI

Conversational ground-freight planning with evidence and verified route decisions.

01 PROBLEM & USE CASE

The project

Turn a shipment request into feasible, ranked routes.

02 DATA & SOURCES

The data

A reproducible network and a versioned document corpus.

03 METHODS

The approach

NLP + hybrid RAG + deterministic optimization.

04 EVALUATION

Test routing correctness, language extraction, retrieval and answer grounding separately.

What is RouteMind AI?

A conversational decision-support prototype for one ground-freight shipment at a time.

START WITH THE NEED

Describe a shipment

Enter the locations, quantity, spending limit, delivery needs and cargo requirements.

UNDERSTAND THE OPTIONS

Compare suitable routes

Review prices, arrival times, transfers and the information supporting the result.

KEEP THE PLANNER IN CONTROL

Explore and decide

Ask a follow-up or a carrier-policy question, then review the result before choosing.

The presentation explains the project, its data, the methods used, and how its correctness and answer quality are evaluated.

Choose a route that meets the shipment’s needs

For logistics coordinators balancing price, timing, capacity and cargo care.

INPUT

Describe the shipment

Use everyday language; clarify missing details before planning.

DECISION SUPPORT

Review the trade-offs

Compare up to three routes, arrival times and cost breakdowns.

INTERACTION

Ask and revise

Change priorities or ask about carrier documents with citations.

AI interprets language. SQL establishes facts. The optimizer searches and verifies routes.

Fragile electronics: Fremont to Austin

15 pallets • $4,000 ceiling • Fixed demo start: November 1, 2026, 06:00 UTC.

Priority	Route	Total cost	Arrival from demo start
Cheapest	Fremont > Phoenix > Austin	$2,100	+40 hours
Fastest	Fremont > Austin (direct)	$3,400	+32 hours

Follow-up: “Make it faster, but keep the budget.”

The revised request preserves unchanged requirements and compares the verified outcomes.

Synthetic, reproducible and explicitly labelled

Operational facts are authored or generated in app/seed.py; no live carrier feed is connected.

Dataset	Size	Contents / provenance
Network hubs	18	US hub names, timezones and authored transfer rules
Carriers / capability flags	10 / 40	Fictional profiles; four explicit capability flags per carrier
Scheduled services	180	Fixed-seed departures, arrivals, capacities and prices
Evidence corpus	58	50 carrier passages + 4 handling notes + 4 planning cases
Quality benchmark	64 NLP / 50 policy	Synthetic labels; separate development/test groups

SQL: operational records | ChromaDB: document vectors | No private customer data or live quotes

Language enters. Verified decisions leave.

LangGraph coordinates the planning workflow; FastAPI serves the web application.

01

Interpret

Rules or typed Gemini/OpenAI

02

Clarify

Validate fields; ask for gaps

03

Retrieve

MiniLM + BM25 carrier evidence

04

Optimize

Bounded scheduled path enumeration

05

Verify

Recompute from authoritative SQL

06

Explain

Ranked plans + cited context

Two stores, two responsibilities

PostgreSQL/SQLite determines eligibility. ChromaDB supplies supporting text; it cannot approve a route.

Separate interpretation from mathematical correctness

Natural-language flexibility is useful; route feasibility must remain reproducible.

INTERPRETATION

NLP and conversation

Extract fields; ask about ambiguity. Apply contextual edits while retaining other constraints.

OPTIMIZATION

Exact bounded search

Enumerate acyclic paths up to 3 legs. Check budget, timing, capacity, capabilities, reliability and stops.

Independent verification recomputes every returned plan from SQL schedules, prices and transfer rules.

Search by meaning and words, then check the evidence

Policy Q&A selects exact source passages; it does not generate authoritative carrier rules.

HYBRID SEARCH

Retrieve

MiniLM semantic search + BM25 keyword search; merge with RRF.

EVIDENCE SAFEGUARDS

Review

Check relevance, effective dates, revisions and reviewed capability conflicts.

GROUNDED OUTPUT

Cite or abstain

Validate source IDs and quotations. Report insufficient or conflicting evidence.

RRF(d) = sum[1 / (60 + rank(d))]

A correct quotation is not proof that it completely answers the question.

Evaluate each layer independently

Software correctness, retrieval quality and live-model quality are different claims.

Layer	Method / metrics	Current status
Routing	Known fixtures + independent exhaustive oracle	Automated correctness checks
NLP extraction	Micro precision/recall/F1; clarification accuracy	Rules measured; live-model quality unmeasured
Retrieval	Recall@5, MRR, nDCG@5; dense vs hybrid	Measured on synthetic queries
Policy answers	Gold-source citation precision; abstention; human review	Runner implemented; live/human review not performed
End-to-end UI	Browser workflows, errors and mobile layouts	Verified alongside DB test suites

Development/test groups are frozen, but templates are shared and labels are public.

Useful regression evidence, with clear limits

Latest verified software run: 107 tests passed on each of SQLite and PostgreSQL.

Quality-suite result	Test: Hash	Test: MiniLM
Offline rules entity F1 / clarification accuracy	1.00 / 1.00	1.00 / 1.00
Answerable policy retrieval recall	0.90	0.90
Relevance-gate accuracy proxy	0.96	0.96
Live-model / human grounding quality	Not measured	Not measured

Shared templates and near-duplicate passages make these tests optimistic.

MiniLM development gate proxy: 0.88 — semantic similarity can still admit irrelevant text.

An auditable planning assistant, with a defined boundary

Working prototype today; independent quality evidence and real integrations are future work.

DELIVERED

Demonstrated

Conversational planning, cited policy passages, comparisons and a read-only explorer.

EVALUATION PRIORITY

Next to validate

Independent labels, live-model evaluation, human grounding and gate calibration.

EXPLICIT LIMITS

Outside today’s scope

Live rates, traffic, bookings, real capacity and production multi-user operation.

Demo: describe a shipment > revise its priority > inspect a policy source.

Repository: github.com/chetsp/RouteMind-AI